

```

1 //Solucionario Lab 2016
2
3 import java.text.SimpleDateFormat;
4
5 // Clase de arranque: arma el escenario y pide el reporte.
6 // 'throws Exception' es obligatorio porque sdf.parse() lanza ParseException.
7 class Principal {
8     public static void main(String[] args) throws Exception {
9
10         // parse() convierte un String en Date usando este patron.
11         SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
12
13         Equipu objEquipu = new Equipu();
14
15         // Se crean los equipos
16         Equipo equipo = new Equipo("HCI-DUXAIT", "Investigaciones en el area de Interaccion Humano-Computador"
17 );
18         // Se crean los miembros del equipo.
19         // FIJATE EN EL TIPO: la variable se declara como Miembro (el padre)
20         // aunque el objeto sea Alumno, Profesor o MiembroExterno. Eso es
21         // polimorfismo: el tipo de la variable no tiene que ser el de la
22         // clase concreta, basta con que sea una superclase suya.
23         Miembro m1 = new Alumno("Juan Perez", sdf.parse("02-11-1992"), "Av. Universitaria 2019", "jperez@pucp.
24 edu.pe", 'M', "20090606", 68.3);
25         Miembro m2 = new Alumno("Viviana Rivasplata", sdf.parse("03-01-1988"), "Av. Bolivar 320", "vrivasplata
26 @pucp.edu.pe", 'F', "20096969", 64.5);
27         Miembro m3 = new Profesor("Andrea Montenegro", sdf.parse("05-08-1974"), "Av. La Marina 1992", "amonten
28 egro@pucp.edu.pe", 'F', "64891", Estado.TC);
29         // Este NO es de la PUCP: no tiene codigo PUCP ni consultarDatos().
30         // Es el caso que el instanceof de Equipo tendra que filtrar.
31         Miembro m4 = new MiembroExterno("Jorge Gonzales", sdf.parse("01-05-1992"), "Av. Pardo y Aliaga 530", "
32 jgonzales@empresa.com", 'M', TipoDedicacion.P);
33
34         // Agregamos los miembros al equipo (los 4 entran por el mismo metodo)
35         equipo.agregarMiembro(m1);
36         equipo.agregarMiembro(m3);
37         equipo.agregarMiembro(m2);
38         equipo.agregarMiembro(m4);
39
40         // Agregamos los equipos al Equipu
41         objEquipu.agregarEquipo(equipo);
42
43         // El reporte solo debe listar a los miembros PUCP.
44         String reporte = objEquipu.consultarMiembrosDeEquipo(0);
45         System.out.print(reporte);
46     }
47 }
48 // Contrato: quien lo implemente sabe devolver sus datos como texto.
49 // Se usa como interfaz y no como clase abstracta porque solo describe
50 // una CAPACIDAD ("ser consultable"), no un estado compartido.
51 interface IConsultable {
52     String consultarDatos();
53 }
54 // Enum: conjunto cerrado de valores validos.
55 // Evita usar String sueltos y typos como "tc" o "T.C.".
56 // TPA = tiempo parcial por asignatura, TC = tiempo completo,
57 // TPC = tiempo parcial por convenio.
58 enum Estado {
59     TPA, TC, TPC
60 }
61 // T = tiempo completo, P = tiempo parcial.
62 enum TipoDedicacion {
63     T, P
64 }
65 import java.util.Date;
66
67 // Raiz de la jerarquia. Guarda lo comun a CUALQUIER miembro,
68 // sea de la PUCP o externo.
69 class Miembro {
70     // static: existe UNA sola copia compartida por toda la clase, no una
71     // por objeto. Sirve de contador para autogenerar codigos correlativos.
72     private static int correlativo = 1;
73
74     private int codigo;
75     private String nombre;
76     private Date fechaNacimiento;
77     private String direccion;
78     private String email;
79     private char sexo;
80
81     // Constructor vacio: permite crear un Miembro sin datos si hiciera falta.

```

```

78     public Miembro() {}
79
80     public Miembro(String nombre, Date fechaNacimiento, String direccion, String email, char sexo) {
81         // correlativo++ asigna el valor actual y RECIEN despues incrementa,
82         // asi el primer miembro queda con codigo 1.
83         this.codigo = correlativo++;
84         this.nombre = nombre;
85         this.fechaNacimiento = fechaNacimiento;
86         this.direccion = direccion;
87         this.email = email;
88         this.sexo = sexo;
89     }
90
91     public String getNombre() {
92         return nombre;
93     }
94 }
95 import java.util.Date;
96
97 // Nivel intermedio: hereda de Miembro Y ADEMÁS implementa una interfaz.
98 // En Java se hereda de UNA sola clase pero se pueden implementar VARIAS
99 // interfaces; por eso 'extends' va antes que 'implements'.
100 //
101 // Es abstracta porque "miembro PUCP" a secas no existe: siempre sera
102 // alumno o profesor. Aporta el codigo PUCP, que el externo no tiene.
103 abstract class MiembroPUCP extends Miembro implements IConsultable {
104     private String codigoPUCP;
105
106     public MiembroPUCP(String nombre, Date fechaNacimiento, String direccion, String email, char sexo, String
107     codigoPUCP) {
108         // super() sube al constructor de Miembro (que ahi asigna el correlativo).
109         super(nombre, fechaNacimiento, direccion, email, sexo);
110         this.codigoPUCP = codigoPUCP;
111     }
112
113     public String getCodigoPUCP() {
114         return codigoPUCP;
115     }
116
117     // Se vuelve a declarar abstracto: esta clase no sabe como presentarse,
118     // lo resuelven Alumno y Profesor cada uno a su manera.
119     public abstract String consultarDatos();
120 }
121 import java.util.Date;
122
123 // Cuelga directamente de Miembro, saltandose MiembroPUCP.
124 // Por eso NO implementa IConsultable y no tiene consultarDatos():
125 // es justamente el objeto que el reporte debe dejar fuera.
126 class MiembroExterno extends Miembro {
127     private TipoDedicacion tipoDedicacion;
128
129     public MiembroExterno(String nombre, Date fechaNacimiento, String direccion, String email, char sexo, Tipo
130     Dedicacion tipoDedicacion) {
131         super(nombre, fechaNacimiento, direccion, email, sexo);
132         this.tipoDedicacion = tipoDedicacion;
133     }
134 }
135 import java.util.Date;
136
137 // Hoja de la jerarquia. Aporta lo unico propio del alumno: el CRAEST.
138 class Alumno extends MiembroPUCP {
139     private double CRAEST;
140
141     public Alumno(String nombre, Date fechaNacimiento, String direccion, String email, char sexo, String codig
142     oPUCP, double CRAEST) {
143         // Cadena de 3 niveles: Alumno -> MiembroPUCP -> Miembro.
144         super(nombre, fechaNacimiento, direccion, email, sexo, codigoPUCP);
145         this.CRAEST = CRAEST;
146     }
147
148     // @Override no es obligatorio, pero avisa en compilacion si la firma
149     // no coincide con la del padre. Conviene siempre ponerlo.
150     @Override
151     public String consultarDatos() {
152         // getCodigoPUCP() viene de MiembroPUCP, getNombre() de Miembro.
153         return "Alumno: " + getCodigoPUCP() + " - " + getNombre() + " - " + CRAEST + "\n";
154     }
155 }
156 import java.util.Date;
157
158 // La otra hoja. Misma estructura que Alumno pero con Estado en vez de CRAEST.
159 class Profesor extends MiembroPUCP {

```

```

157     private Estado estado;
158
159     public Profesor(String nombre, Date fechaNacimiento, String direccion, String email, char sexo, String codigoPUCP, Estado estado) {
160         super(nombre, fechaNacimiento, direccion, email, sexo, codigoPUCP);
161         this.estado = estado;
162     }
163
164     // Mismo nombre de metodo que en Alumno, distinto resultado.
165     // Esa es la esencia del polimorfismo.
166     @Override
167     public String consultarDatos() {
168         return "Profesor: " + getCodigoPUCP() + " - " + getNombre() + " - " + estado + "\n";
169     }
170 }
171 import java.util.ArrayList;
172 import java.util.List;
173
174 // Un equipo de investigacion con sus miembros.
175 class Equipo {
176     private String nombre;
177     private String interes;
178     // Lista del tipo del PADRE: guarda alumnos, profesores y externos juntos.
179     private List<Miembro> miembros;
180     private Equipu Equipu;
181
182     public Equipo(String nombre, String interes) {
183         this.nombre = nombre;
184         this.interes = interes;
185         // Sin este new, la lista vale null y el primer add() revienta.
186         this.miembros = new ArrayList<>();
187     }
188
189     public void agregarMiembro(Miembro miembro) {
190         this.miembros.add(miembro);
191     }
192
193     // AQUI ESTA EL PUNTO CLAVE DEL LAB.
194     // La lista es de Miembro, y Miembro no tiene consultarDatos(): solo
195     // lo tienen los MiembroPUCP. Entonces hay que hacer dos cosas:
196     // 1) instanceof -> preguntar si ese objeto ES de ese tipo
197     // 2) (MiembroPUCP)m -> downcast, decirle al compilador que lo trate
198     // como MiembroPUCP para poder llamar al metodo
199     // Castear sin preguntar antes lanza ClassCastException en ejecucion.
200     // El MiembroExterno cae por el filtro y no sale en el reporte.
201     public String consultarDatosMiembrosPUCP() {
202         String cadena = "";
203         for (Miembro m : miembros) {
204             if (m instanceof MiembroPUCP) {
205                 cadena += ((MiembroPUCP) m).consultarDatos();
206             }
207         }
208         return cadena;
209     }
210 }
211 import java.util.ArrayList;
212 import java.util.List;
213
214 // Sistema contenedor: agrupa todos los equipos.
215 // Delega: no sabe consultar miembros, le pide al Equipo que lo haga.
216 class Equipu {
217     private List<Equipo> equipos;
218     private List<Miembro> ejecutivos;
219
220     public Equipu() {
221         this.equipos = new ArrayList<>();
222         this.ejecutivos = new ArrayList<>();
223     }
224
225     public void agregarEquipo(Equipo equipo) {
226         this.equipos.add(equipo);
227     }
228
229     public String consultarMiembrosDeEquipo(int indice) {
230         // get(indice) saca el equipo de la lista y le pasa la pelota a el.
231         return equipos.get(indice).consultarDatosMiembrosPUCP();
232     }
233 }

```